

schema

Java ds-api counterpart in :java: `com.feedzai.commons.dsapi.Schema` .

Field module-attribute

```
Field = autoclass(DS_API_CLASSPATH + 'Field')
```

FieldType module-attribute

```
FieldType = autoclass('com.feedzai.pulse.esp.FieldType')
```

logger module-attribute

```
logger = getLogger(__name__)
```

Schema

```
Schema(pyjnius_java)
```

Bases: `JavaWrapper`

Represents a Schema in Pulse.

subclasses class-attribute instance-attribute

```
subclasses: List[JavaWrapper] = []
```

add_field

```
add_field(field: FieldTemplate, index: int | None = None)
```

Adds a new field to the schema.

Parameters:

Name	Type	Description	Default
<code>field</code> 	<code>FieldTemplate</code>	The new field, which is created according to the field template and added to the schema.	<i>required</i>
<code>index</code> 	<code>int</code>	The index of the new field in the Schema's list of fields. For example, if <code>index = 0</code> , the new schema will be added in the first position. If no index is specified, the field is appended to the end of the Schema.	<code>None</code>

Examples:

```
>>> app = Feedzai().app(name="sample")
>>> schema = app.pullSchemaByDesc(name="target-schema")
>>> schema.add_field(field=FieldTemplate(
...     "numeric_field",
...     NativeType.FLOAT,
...     NumericalField())
```

```
... )
>>> schema.add_field(field=
...     FieldTemplate("categorical_field",
...         NativeType.STRING,
...         CategoricalField({"value1", "value2"}))
... ), index=0)
```

add_fields [¶](#)

```
add_fields(fields: list[FieldTemplate])
```

Adds a list of new fields to the schema. The fields are added at the end of the schema following the order of the provided list.

Parameters:

Name	Type	Description	Default
fields ¶	list of <code>FieldTemplate</code>	The list of new fields, which are created according to the corresponding field template and added to the schema.	<i>required</i>

Examples:

```
>>> app = Feedzai().app(name="sample")
>>> schema = app.pullSchemaByDesc(name="target-schema")
>>> fields = [
...     FieldTemplate("numeric_field", NativeType.FLOAT, NumericalField()),
...     FieldTemplate(
...         "timestamp_msec_field",
...         NativeType.LONG,
...         DateTimeField(TimestampFormat("MILLISECONDS")))
...     ),
... ]
>>> schema.add_fields(fields)
```

create classmethod [¶](#)

```
create(app, name)
```

Create a new Schema in `app` with the given `name` .

Parameters:

Name	Type	Description	Default
app ¶	<code>App</code>	The App where the schema will be created.	<i>required</i>
name ¶	<code>str</code>	The name of the schema. Has to start with a non-special character.	<i>required</i>

Returns:

Type	Description
<code>Schema</code>	Schema object corresponding to the Pulse schema created.

desc_to_name [¶](#)

```
desc_to_name(desc: str) -> str
```

Return the internal (sanitized) representation of a field name as it appears in the UI. If `desc` is already the sanitized version of the name, that is returned instead.

Raises:

Type	Description
<code>ValueError</code>	When <code>desc</code> is not part of the schema.

duplicate [↗](#)

```
duplicate(new_name: str | None = None) -> Schema
```

Duplicates the schema.

Parameters:

Name	Type	Description	Default
<code>new_name</code> ↗	<code>str</code>	Name of the schema duplicate, by default will be the name of the source schema followed by <code>'_duplicate'</code> .	<code>None</code>

Returns:

Type	Description
<code>Schema</code>	The schema duplicate.

name_to_desc [↗](#)

```
name_to_desc(name: str) -> str
```

Return the description of a field as it appears in Pulse UI from its internal name representation.

Raises:

Type	Description
<code>ValueError</code>	When <code>name</code> is not part of the schema.

remove_fields [↗](#)

```
remove_fields(fields_to_remove: set[str])
```

Removes a list of fields from the schema.

Parameters:

Name	Type	Description	Default
<code>fields_to_remove</code> ↗	<code>Set[str]</code>	A set with the names of the fields to remove.	<i>required</i>

Raises:

Type	Description
<code>ValueError</code>	If at least one of the provided field names does not exist in the schema.

reorder_fields [¶](#)

```
reorder_fields(order: tuple[int, ...])
```

Reorders the fields in the schema according to the specified list of indices. The new order is specified with a tuple of integers where each element is the index of the field in the original list of fields.

Parameters:

Name	Type	Description	Default
<code>order</code> ¶	<code>tuple of int</code>	Tuple with the field indices in the new order.	<i>required</i>

Raises:

Type	Description
<code>(ValueError, TypeError)</code>	If the specified order is not valid.

Examples:

```
>>> app = Feedzai().app(name="sample")
>>> schema = app.pullSchemaByDesc(name="four-field-schema")
>>> schema.reorder_fields((1, 0, 2, 3))
>>> schema.reorder_fields((3, 2, 1, 0))
```

reorder_fields_by_name [¶](#)

```
reorder_fields_by_name(ordered_names: tuple[str, ...])
```

Reorders the fields in the schema according to the specified list of names.

Parameters:

Name	Type	Description	Default
<code>ordered_names</code> ¶	<code>tuple[str, ...]</code>	Tuple with the field names in the new order. The tuple must be an exact match to the schema field names.	<i>required</i>

Raises:

Type	Description
<code>(ValueError, TypeError)</code>	If the specified list of names is not an exact match to the schema field names.

Examples:

```
>>> app = Feedzai().app(name="sample")
>>> schema = app.pullSchemaByDesc(name="four-field-schema")
>>> schema.reorder_fields_by_name(("field2", "field1", "field3", "field4"))
>>> schema.reorder_fields_by_name(("field4", "field3", "field2", "field1"))
```

get_internal_name

```
get_internal_name(desc: str, schema: Schema | None = None) -> str
```

Return the internal (sanitized) name of a field.

This is a utility function to improve the UX of ds-api. Through ds_api the user should only care about the names as they appear in the UI ('desc').

Note

If a name is not found or the schema is not passed, this function acts as a no-op.

get_internal_names

```
get_internal_names(names: _T, schema: Schema | None = None) -> _T
```

Return a sequence of internal (sanitized) names based on names as they appear in the UI.

Examples:

```
>>> get_internal_names({"A Fancy Name here", "and another there"}, schema)
{"a_fancy_name_here", "and_another_there"}
```

See Also

[ds_api.schema.get_internal_name](#)